

1) Camera-ready abstract (tightened)

We present a transformer reformulation grounded in the Quaternionic Recursive Harmonic (ΨQRH) framework. The model replaces standard attention/FFN blocks with quaternionic state evolution and FFT-based spectral attention, plus Leech-lattice–inspired parameter grouping for error resilience. A PyTorch reference implementation demonstrates $O(n \log n)$ attention behavior, competitive perplexity on WikiText-103/C4 (small-scale runs), and throughput/memory gains on A100s in our environment. We include ablations (real vs complex vs quaternion), spectral filtering on/off, and stability checks. Results are preliminary but reproducible with the provided scripts.

Responsible claims: All speed/PPL numbers in this README are from our hardware and training budgets; they may differ on other stacks. Use the provided scripts to reproduce and report.

2) Minimal, correct PyTorch block (drop-in)

```
# qrh_layers.py
import torch
import torch.nn as nn
import torch.fft as fft

def split_quat(x):
    # x: [B, T, 4*D] -> tuple of 4 tensors [B,T,D]
    B,T,FD = x.shape
    D = FD // 4
    return x[..., :D], x[..., D:2*D], x[..., 2*D:3*D], x[..., 3*D:4*D]

def hamilton_product(q1, q2):
    # q1,q2 each tuple (r,i,j,k) with shapes [B,T,D]
    r1,i1,j1,k1 = q1; r2,i2,j2,k2 = q2
    r = r1xr2 - i1xi2 - j1xj2 - k1xk2
    i = r1xi2 + i1xr2 + j1xk2 - k1xj2
    j = r1xj2 - i1xk2 + j1xr2 + k1xi2
    k = r1xk2 + i1xj2 - j1xi2 + k1xr2
    return (r,i,j,k)

def quat_cat(q):
    # q tuple -> cat back to [B,T,4*D]
```

```
return torch.cat(q, dim=-1)
```

```
class SpectralFilter(nn.Module):
```

```
    """Log-phase filter:  $F(k) = \exp(i * \alpha * \text{atan}(\ln(|k| + \epsilon)))$ ."""
```

```
    def __init__(self, alpha=1.0, eps=1e-10):
```

```
        super().__init__()
```

```
        self.alpha = nn.Parameter(torch.tensor(float(alpha)))
```

```
        self.eps = eps
```

```
    def forward(self, Kc):
```

```
        # Kc: complex tensor (freq domain), shape [..., D]
```

```
        mag = torch.clamp(torch.abs(Kc), min=self.eps)
```

```
        phase = self.alpha * torch.atan(torch.log(mag))
```

```
        return Kc * torch.exp(1j * phase)
```

```
class QRHAttention(nn.Module):
```

```
    """
```

```
    Quaternionic spectral attention ( $O(n \log n)$ ):
```

```
    1) linear proj -> quaternion states
```

```
    2) frequency-domain mixing + log-phase filtering
```

```
    3) inverse FFT back to time domain
```

```
    """
```

```
    def __init__(self, dim, heads=8):
```

```
        super().__init__()
```

```
        assert dim % heads == 0
```

```
        self.dim = dim
```

```
        self.heads = heads
```

```
        self.head_dim = dim // heads
```

```
        self.to_qkv = nn.Linear(dim, 3 * 4 * dim, bias=False) # 4D quaternion per Q/K/V
```

```
        self.out = nn.Linear(4 * dim, dim, bias=False)
```

```
        self.filter = SpectralFilter(alpha=1.0)
```

```
    def _proj_quat(self, x):
```

```
        # x: [B,T,D] -> (Q,K,V) each as tuples of 4 parts [B,T,heads*head_dim]
```

```
        B,T,D = x.shape
```

```
        qkv = self.to_qkv(x) # [B,T,12D]
```

```
        qkv = qkv.view(B, T, 3, 4, D) # split Q/K/V, quaternion parts
```

```
        Q = tuple(qkv[:, :, 0, i, :] for i in range(4))
```

```
        K = tuple(qkv[:, :, 1, i, :] for i in range(4))
```

```
        V = tuple(qkv[:, :, 2, i, :] for i in range(4))
```

```
        return Q,K,V
```

```

def forward(self, x):
    # x: [B,T,D]
    B,T,D = x.shape
    Q,K,V = self._proj_quat(x) # quaternion tuples
    # Simple quaternion "similarity": elementwise Hamilton mix in freq domain
    # Move to frequency domain along sequence axis (T)
    def fft1(q):
        return tuple(fft.fft(qp, dim=1) for qp in q) # complex
    def ifft1(qc):
        return tuple(fft.ifft(qp, dim=1) for qp in qc) # complex back

    Qf, Kf, Vf = fft1(Q), fft1(K), fft1(V)

    # Spectral mixing: use Hamilton product in frequency domain (component-wise)
    # Option: combine Q/K, then apply phase filter, then apply to V
    QKf = tuple(q + k for q,k in zip(Qf, Kf)) # simple mix; replace with learned mix if desired

    # Apply log-phase filter to each component
    QKf_filt = tuple(self.filter(comp) for comp in QKf)

    # Apply filtered "attention" to V (here: Hamilton product surrogate)
    mixed = tuple(qkf × vf for qkf, vf in zip(QKf_filt, Vf))

    # Back to time domain
    mix_t = ifft1(mixed) # complex
    # Take real part (phase already encoded)
    mix_real = tuple(torch.real(m) for m in mix_t) # 4 tensors [B,T,D]
    y = quat_cat(mix_real) # [B,T,4D]
    return self.out(y) # [B,T,D]

```

```

class QRHFeedForward(nn.Module):

```

```

    """Harmonic evolution via spectral filter + learned unit quaternion rotation."""

```

```

    def __init__(self, dim, mult=4):

```

```

        super().__init__()

```

```

        self.inp = nn.Linear(dim, 4*dim, bias=False)

```

```

        self.rot = nn.Linear(4*dim, 4*dim, bias=False) # surrogate for unit rotation (can orthonormalize)

```

```

        self.out = nn.Linear(4*dim, dim, bias=False)

```

```

        self.filter = SpectralFilter(alpha=1.0)

```

```

    def forward(self, x):

```

```

z = self.xinp(x) # [B,T,4D]
r,i,j,k = split_quat(z)
# FFT along T
rc = fft.fft(r, dim=1); ic = fft.fft(i, dim=1); jc = fft.fft(j, dim=1); kc = fft.fft(k, dim=1)
rc,ic,jc,kc = [self.filter(comp) for comp in (rc,ic,jc,kc)]
r2,i2,j2,k2 = [torch.real(fft.ifft(comp, dim=1)) for comp in (rc,ic,jc,kc)]
z2 = torch.cat([r2,i2,j2,k2], dim=-1)
z3 = self.rot(z2)
return self.out(z3)

```

Notes

- Shapes are explicit; FFT is along sequence axis (dim=1).
- SpectralFilter is learnable (alpha).
- Quaternion operations are implemented without exotic deps.
- This is *reference* code; you can swap the simple spectral mix with a learned kernel.

3) Tiny eval harness (so claims are reproducible)

```

# run_wikitext_small.py
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from datasets import load_dataset
from transformers import AutoTokenizer
from qrh_layers import QRHAttention, QRHFeedForward

```

```

class Block(nn.Module):
    def __init__(self, dim, heads=8):
        super().__init__()
        self.attn = QRHAttention(dim, heads)
        self.ffn = QRHFeedForward(dim)
        self.xnorm1 = nn.LayerNorm(dim)
        self.xnorm2 = nn.LayerNorm(dim)
    def forward(self, x):
        x = x + self.attn(self.xnorm1(x))
        x = x + self.ffn(self.xnorm2(x))
        return x

```

```

class QRHTransformer(nn.Module):

```

```

def __init__(self, vocab, dim=512, depth=6, heads=8):
    super().__init__()
    self.emb = nn.Embedding(vocab, dim)
    self.blocks = nn.ModuleList([Block(dim, heads) for _ in range(depth)])
    self.ln = nn.LayerNorm(dim)
    self.out = nn.Linear(dim, vocab)
def forward(self, ids):
    x = self.emb(ids)
    for b in self.blocks: x = b(x)
    x = self.ln(x)
    return self.out(x)

def main():
    tok = AutoTokenizer.from_pretrained("gpt2")
    ds = load_dataset("wikitext", "wikitext-2-raw-v1", split="train[:1%]")
    def encode(ex): return {"ids": tok(ex["text"], truncation=True, padding="max_length",
max_length=256)["input_ids"]}
    ds = ds.map(encode, remove_columns=ds.column_names)
    dl = DataLoader(ds, batch_size=8, shuffle=True)
    model = QRHTransformer(vocab=tok.vocab_size).cuda()
    opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
    lossf = nn.CrossEntropyLoss()
    for step, batch in enumerate(dl):
        ids = torch.tensor(batch["ids"]).cuda()
        logits = model(ids[:, :-1])
        loss = lossf(logits.reshape(-1, logits.size(-1)), ids[:, 1:].reshape(-1))
        opt.zero_grad(); loss.backward(); opt.step()
        if step % 10 == 0:
            ppl = torch.exp(loss.detach().cpu())
            print(f"step {step} loss {loss.item():.3f} ppl {ppl:.2f}")
        if step == 50: break

if __name__ == "__main__":
    main()

```

- This won't replicate your full tables, but it *does* let others observe stable training, speed, and memory footprints immediately.
 - Provide a requirements.txt with exact torch/cuda/datasets/transformers pins.

4) Results & ablations (reframe)

Keep your tables, but mark them as **"our environment"** and add a script pointer for reproduction:

- scripts/benchmark_speed_mem.py (reports tokens/s, peak GB via torch.cuda.max_memory_allocated).
- scripts/ablation_real_complex_quat.py
- scripts/ablation_filter_on_off.py

Also add a **"Known variance"** line: throughput varies with FFT kernel and sequence length; report n, seq_len, precision, GPU model.

5) "Truth-Vectoring" guardrail paragraph (README)

We use a *Truth-Vectoring* discipline: every claim in this README maps to a script in /scripts or a figure produced by /validation. If a reader cannot reproduce a number within 10–15% on comparable hardware, we consider that a bug in our reporting or environment notes, not a rebuttal. Please open an issue with logs.

6) Release checklist (quick)

- qrh_layers.py (clean, importable)
 - minimal model QRHTransformer
 - run_wikitext_small.py (smoke test)
 - /scripts for speed/mem and ablations
 - /validation producing the figs you cite
 - requirements.txt with exact pins
 - LICENSE (GPLv3 as you stated)
 - "Responsible claims" box + Truth-Vectoring note
 - Mark forward-looking refs (e.g., "Padilha 2025") as **in prep / preprint** unless you have a DOI
-

7) Where to push next (high-impact, low risk)

- **Numerics:** add unit tests for norm preservation and gradient flow under AMP (fp16/bf16).
- **Quantization note:** document current incompatibilities (int8) and a roadmap.
- **Scale:** one curated run at ~300–500M params on WikiText-103 with logs + wandb

artifact.

Appendix — Implementation Details (Ψ QRH Transformer)

A. Model Specs

- **Embedding dim:** 512
- **Heads:** 8
- **Layers:** 6
- **FFN dim:** 2048
- **LR:** 1e-4
- **Batch size:** 32
- **Training steps:** 100k

B. Compute & Environment

- **GPUs:** 4 × A100 (40 GB)
- **Peak model memory:** ~7.3 GB (attention path)
- **Train time (ref env):** ~72 h
- **Stack:** PyTorch 2.0, CUDA 11.7, Python 3.10+

⚠ Throughput/footprint vary with sequence length, FFT kernel, precision, and dataloader I/O. See “Repro & Logging”.

C. Repository Layout

quaternionic-transformer/

```
├── models/
│   ├── attention.py    # Spectral (FFT) attention w/ quaternion states
│   ├── embeddings.py   # Quaternionic embeddings / mapping
│   └── ffn.py          # Harmonic FFN (spectral filter + rotation)
├── utils/
│   ├── quaternion.py   # Hamilton ops, split/pack, unit rotations
│   └── lattice.py      # Leech lattice / Golay G24 helpers
├── scripts/
│   ├── run_wikitext_small.py  # smoke test / tiny PPL run
│   ├── benchmark_speed_mem.py # tokens/s + peak GB
│   ├── ablation_real_complex_quat.py
│   └── ablation_filter_on_off.py
├── validation/
│   ├── simple_validation_test.py
│   ├── robust_validation_test.py
│   └── comprehensive_integration_test.py
```

```
|  └── figures/ # autogenerated plots
└── configs/
    └── base.yaml
```

configs/base.yaml (example)

model:

```
d_model: 512
n_heads: 8
n_layers: 6
d_ff: 2048
spectral_alpha: 1.0
seq_len: 1024
```

train:

```
steps: 100000
batch_size: 32
lr: 1.0e-4
weight_decay: 0.01
grad_clip: 1.0
precision: bf16 # {fp32,bf16,fp16}
```

data:

```
dataset: wikitext-103
max_length: 1024
num_workers: 8
```

system:

```
gpus: 4
seed: 1337
log_every: 50
eval_every: 2000
save_every: 5000
```

requirements.txt (pin as used)

```
torch==2.0.1
torchvision==0.15.2
transformers==4.41.0
datasets==2.19.0
numpy==1.26.4
```


scipy==1.12.0
matplotlib==3.8.4
seaborn==0.13.2
pyyaml==6.0.1
tqdm==4.66.4

D. Fractal Module Summary (needle_fractal_dimension.py)

Purpose: Generate fractal point clouds (IFS), estimate dimension (box-counting + spectral), and render demos used in the paper.

Key parts

- **FractalGenerator**
- Adds affine contractive transforms (add_transform), generates attractors via chaos game (generate(dim=2/3)).
- calculate_fractal_dimension(method="box"|"spectral"):
- **Box-counting:** normalize to unit cube; grid at ϵ ; count non-empty boxes $N(\epsilon)$; slope of $\log N$ vs $\log(1/\epsilon) \rightarrow D$.
- **Spectral (2D):** density grid $\rightarrow$ 2D FFT $\rightarrow$ radial average $\rightarrow$ fit power law $P(k) \sim k^{-\beta}$ $\rightarrow$ map $\beta \rightarrow D$.
- **LaserPulseSimulator** (experimental)
- Complex quadratic-chirp pulse; scan over fractal; analyze response spectrum (prototype for physical probing).
- **Visualization**
- plot_box_counting_demo() $\rightarrow$ needle_box_counting_demo.png
- plot_spectral_analysis_demo() $\rightarrow$ needle_spectral_analysis_demo.png
- **main()**
- Builds Sierpinski IFS, generates points, computes D by both methods, prints side-by-side vs theory, writes needle_results.png.

E. 4D Unitary Layer (Ψ QRH Core)

Group structure: $SO(4) \cong (SU(2) \times SU(2)) / \mathbb{Z}_2$.

Operation: $v' = q \{ \text{left} \} ; v ; q_{\text{right}}^{\dagger}$ (Hamilton products).

Spectral regularization: $H(k) = \exp\{ i \alpha \cdot \log(|k| + \epsilon) \}$ (or log-phase via atan-ln variant).

Path: FFT along sequence $\rightarrow$ filter phase $\rightarrow$ iFFT $\rightarrow$ learned quaternionic rotation $\rightarrow$ residual.

Receipts (gate controller)

- Orthogonality error E_{orth}
- Energy ratio R_{energy}

- Drift angle θ_{drift}

Gate outcomes: DELIVER (within tolerances), CLARIFY (mid-confidence), ABSTAIN (hazard brake).

Scaling snapshot (illustrative microbench):

Embed	Fwd (ms)	Mem (KB)	Energy ratio
16	2.1	8.4	0.98
32	4.7	33.6	0.97
64	11.2	134.4	0.96
128	28.9	537.6	0.95

F. Validation Suite (what passes / how)

- **Quaternion ops:** norm preservation, Hamilton composition (assoc. within numeric tolerance), non-commutativity checks.
- **Spectral filter:** unit magnitude, stable phase response across $\alpha \in [0.1, 3.0]$.
- **Energy conservation:** $\|x\|_2 \approx \|y\|_2$ ($\pm 5\%$) on synthetic sweeps.
- **Integration stability:** residual stacks under AMP (bf16/fp16) with bounded loss spikes.
- **Fractal analysis:** Sierpinski & Cantor; spectral vs box-counting within expected discretization error.

Run:

```
python validation/simple_validation_test.py
python validation/robust_validation_test.py
python validation/comprehensive_integration_test.py
```

G. Quick Repro (tiny)

```
python scripts/run_wikitext_small.py \
  --seq_len 256 --batch_size 8 --steps 50 --precision bf16
# prints step-wise loss & PPL; confirms stable training + FFT path works
```

H. Repro & Logging (so results travel)

Always log:

- **Hardware:** GPU model/count, driver, CUDA, cuDNN

- **Precision:** fp32 / bf16 / fp16
- **Seq len, batch size, grad accum**
- **Throughput:** tokens/s (model.forward tokens / wall-clock)
- **Memory:** torch.cuda.max_memory_allocated()
- **PPL metric:** dataset slice, tokenizer, max_length

We consider a claim **reproduced** if your tokens/s and peak GB are within $\pm 15\%$ on comparable hardware and your small-run PPL trend matches (absolute values will differ on tiny budgets).

I. Known Limits / TODO

- Sensitivity to α (spectral filter) and rotation params $\Rightarrow$ add auto-tuning.
- Quantization path (int8) not yet supported for custom ops; roadmap planned.
- Very large models ($\gg 500$ M) untested; needs kernel/comm optimizations.